LAB EXPERIMENTS

Classical Search Problems in Python

Title	:	Lab Experiments
Course Name	:	Artificial Intelligence
Course Code	:	1709
Submitted By	:	P. Gururamcharan
Registration No.	:	1925260193
Under Supervision of	:	T. Herlin

Exp. No.	Experiment	CO	Technique
1	8-Puzzle Problem	CO1	BFS
2	8-Queens Problem	CO1	Backtracking
3	Water Jug Problem	CO1	BFS
4	Missionaries and Cannibals Problem	CO1	BFS

Experiment 1: 8-Puzzle Problem

Course Outcome: **CO1**

Aim

To write a Python program to solve the 8-Puzzle problem using Breadth First Search (BFS).

Software / Requirements

Python 3 (any version 3.6 or later) with the standard library only (collections.deque). Any editor or IDE such as IDLE, VS Code or PyCharm.

Theory

The 8-Puzzle consists of a 3x3 board with eight numbered tiles and one blank space. A tile adjacent to the blank can slide into it. The aim is to rearrange the tiles from a given initial state into the goal state (1 2 3 / 4 5 6 / 7 8 blank).

Each arrangement of the board is a **state**. Sliding a tile is an **operator** that moves from one state to another. BFS explores states level by level, so the first time it reaches the goal it has found a solution with the minimum number of moves.

A puzzle is solvable only when the number of inversions in the tile sequence (ignoring the blank) is even. The program checks this first.

Algorithm

1. Represent the board as a tuple of 9 numbers (0 = blank).
2. Check solvability by counting inversions. If odd, stop: not solvable.
3. Put the start state in a queue and mark it visited.
4. Remove the first state from the queue. If it equals the goal, stop and print the path.
5. Generate all neighbours by moving the blank Up, Down, Left, Right (within the board).
6. Add each unvisited neighbour to the queue with the path so far. Repeat from step 4.

Procedure

1. Open a Python editor / IDE and create a new file named **exp1_8puzzle.py**.
2. Type the program given below and save the file.
3. Open a terminal or command prompt in the folder containing the file.
4. Run the program with the command **python exp1_8puzzle.py**.
5. Observe the output, verify that it satisfies the problem conditions and note the result.

Program - Experiment 1

```
# Experiment 1: 8-Puzzle problem using Breadth First Search (BFS)
# The board is a list of 9 numbers read row by row. 0 means the blank space.

goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]


def show(board):
    # print the board as a 3 x 3 grid
    for i in range(0, 9, 3):
        row = ""
        for x in board[i:i + 3]:
            if x == 0:
                row = row + "_ "
            else:
                row = row + str(x) + " "
        print(row)
    print("")


def next_boards(board):
    # return all boards that can be made by sliding one tile into the blank
    result = []
    b = board.index(0)          # position of the blank (0 to 8)
    r = b // 3                  # row of the blank
    c = b % 3                   # column of the blank

    if r > 0:                   # blank can move up
        result.append(("Up", b - 3))
    if r < 2:                   # blank can move down
        result.append(("Down", b + 3))
    if c > 0:                   # blank can move left
        result.append(("Left", b - 1))
    if c < 2:                   # blank can move right
        result.append(("Right", b + 1))

    boards = []
    for move, pos in result:
        new = board[:]          # make a copy
        new[b] = new[pos]       # swap blank with the tile
        new[pos] = 0
        boards.append((move, new))
    return boards


def solvable(board):
    # a 3 x 3 puzzle is solvable only if the number of inversions is even
    tiles = [x for x in board if x != 0]
    count = 0
    for i in range(len(tiles)):
        for j in range(i + 1, len(tiles)):
            if tiles[i] > tiles[j]:
                count = count + 1
    return count % 2 == 0


def solve(start):
    # queue holds [board, path]; path is the list of (move, board) so far
    queue = [[start, []]]
    visited = [start]

    while len(queue) > 0:
        item = queue.pop(0)
        board = item[0]
        path = item[1]

        if board == goal:
            return path

        for move, new in next_boards(board):
            if new not in visited:
                visited.append(new)
                queue.append([new, path + [(move, new)]])
    return None


# ---------------- main program ----------------
start = [1, 2, 3,
         4, 0, 6,
         7, 5, 8]
```

```
print("Initial state:")
show(start)

if not solvable(start):
    print("This puzzle cannot be solved.")
else:
    path = solve(start)
    print("Solved in " + str(len(path)) + " moves")
    print("")
    step = 1
    for move, board in path:
        print("Step " + str(step) + ": move blank " + move)
        show(board)
        step = step + 1
```

Output

```
Initial state:
1 2 3
4 _ 6
7 5 8

Solved in 2 moves

Step 1: move blank Down
1 2 3
4 5 6
7 _ 8

Step 2: move blank Right
1 2 3
4 5 6
7 8 _
```

Result

The 8-Puzzle was solved using BFS and the shortest sequence of moves to the goal state was displayed.

Experiment 2: 8-Queens Problem

Course Outcome: **CO1**

Aim

To write a Python program to place 8 queens on an 8x8 chessboard so that no two queens attack each other, using Backtracking.

Software / Requirements

Python 3 (any version 3.6 or later) with the standard library only (collections.deque). Any editor or IDE such as IDLE, VS Code or PyCharm.

Theory

A queen attacks any piece in the same row, same column or same diagonal. The 8-Queens problem asks for a placement of 8 queens on a chessboard such that none attacks another.

Backtracking places queens one row at a time. For each row it tries every column; if the square is safe it moves on to the next row, and if no column works it goes back (backtracks) to the previous row and tries the next column.

The board is stored as a list where board[row] is the column of the queen in that row. This means two queens can never share a row. A square (row, col) is unsafe if an earlier queen has the same column or if the difference of rows equals the difference of columns (same diagonal). The problem has 92 distinct solutions.

Algorithm

1. Start with row 0 and an empty board.
2. If row = 8, all queens are placed: save the solution.
3. For each column 0 to 7, check whether placing a queen at (row, column) is safe.
4. If safe, place the queen and recursively solve for row + 1.
5. After returning, remove the queen (backtrack) and try the next column.
6. Print the number of solutions and the first solution as a board.

Procedure

1. Open a Python editor / IDE and create a new file named **exp2_8queens.py**.
2. Type the program given below and save the file.
3. Open a terminal or command prompt in the folder containing the file.
4. Run the program with the command **python exp2_8queens.py**.
5. Observe the output, verify that it satisfies the problem conditions and note the result.

Program - Experiment 2

```
# Experiment 2: 8-Queens problem using Backtracking
# board[row] stores the column of the queen placed in that row.

N = 8
solutions = []


def is_safe(board, row, col):
    # check the queens already placed in the rows above
    for r in range(row):
        c = board[r]
        if c == col:                        # same column
            return False
        if abs(c - col) == abs(r - row):    # same diagonal
            return False
    return True


def place(board, row):
    if row == N:                            # all 8 queens placed
        solutions.append(board[:])
        return
    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col                # place the queen
            place(board, row + 1)           # go to the next row
            board[row] = -1                 # backtrack (remove the queen)


def show(board):
    for r in range(N):
        line = ""
        for c in range(N):
            if board[r] == c:
                line = line + "Q "
            else:
                line = line + ". "
        print(line)
    print("")


# ---------------- main program ----------------
place([-1] * N, 0)

print("Total solutions for 8-Queens: " + str(len(solutions)))
print("")
print("First solution:")
print("")
show(solutions[0])
print("Queen column in each row: " + str(solutions[0]))
```

Output

```
Total solutions for 8-Queens: 92

First solution:

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

Queen column in each row: [0, 4, 7, 5, 2, 6, 1, 3]
```

Result

The 8-Queens problem was solved using backtracking. 92 solutions were found and the first one was displayed.

Experiment 3: Water Jug Problem

Course Outcome: **CO1**

Aim

To write a Python program to measure an exact quantity of water using two jugs of different capacities (4 L and 3 L, target 2 L) using Breadth First Search.

Software / Requirements

Python 3 (any version 3.6 or later) with the standard library only (collections.deque). Any editor or IDE such as IDLE, VS Code or PyCharm.

Theory

There are two jugs with no measuring marks. Only these operations are allowed: fill a jug completely, empty a jug completely, or pour water from one jug into the other until the source is empty or the destination is full.

A state is the pair (a, b) = litres in jug A and jug B. The start state is (0, 0). Every operation moves from one state to another, so the problem becomes a search over a state space, and BFS finds the shortest sequence of operations that leaves the target amount in either jug.

A solution exists only if the target is not greater than the larger jug and is a multiple of the GCD of the two capacities.

Algorithm

1. Start at state (0, 0) and put it in a queue.
2. Remove a state (a, b). If a or b equals the target, print the path and stop.
3. Generate the six next states: fill A, fill B, empty A, empty B, pour A into B, pour B into A.
4. Add every state not seen before to the queue along with its path.
5. Repeat until the target is reached or the queue becomes empty (no solution).

Procedure

1. Open a Python editor / IDE and create a new file named **exp3_waterjug.py**.
2. Type the program given below and save the file.
3. Open a terminal or command prompt in the folder containing the file.
4. Run the program with the command **python exp3_waterjug.py**.
5. Observe the output, verify that it satisfies the problem conditions and note the result.

Program - Experiment 3

```
# Experiment 3: Water Jug problem using Breadth First Search (BFS)
# A state is (a, b) = litres of water in jug A and jug B.

cap_a = 4       # capacity of jug A
cap_b = 3       # capacity of jug B
target = 2      # litres we want in either jug


def next_states(a, b):
    # all states reachable in one operation, with a description of it
    pour_ab = min(a, cap_b - b)     # water that can move from A to B
    pour_ba = min(b, cap_a - a)     # water that can move from B to A
    return [
        ("Fill jug A", (cap_a, b)),
        ("Fill jug B", (a, cap_b)),
        ("Empty jug A", (0, b)),
        ("Empty jug B", (a, 0)),
        ("Pour A into B", (a - pour_ab, b + pour_ab)),
        ("Pour B into A", (a + pour_ba, b - pour_ba)),
    ]


def solve():
    start = (0, 0)
    queue = [[start, []]]           # each item: [state, steps so far]
    visited = [start]

    while len(queue) > 0:
        item = queue.pop(0)
        state = item[0]
        steps = item[1]
        a = state[0]
        b = state[1]

        if a == target or b == target:
            return steps

        for action, new in next_states(a, b):
            if new not in visited:
                visited.append(new)
                queue.append([new, steps + [(action, new)]])
    return None


# ---------------- main program ----------------
print("Jug A = " + str(cap_a) + " L, Jug B = " + str(cap_b) +
      " L, Target = " + str(target) + " L")
print("")

steps = solve()
if steps is None:
    print("No solution possible.")
else:
    print("Start                 -> (0, 0)")
    n = 1
    for action, state in steps:
        print("Step " + str(n) + ": " + action.ljust(15) + "-> " + str(state))
        n = n + 1
    print("")
    print("Target reached in " + str(len(steps)) + " steps.")
```

Output

```
Jug A = 4 L, Jug B = 3 L, Target = 2 L

Start                  -> (0, 0)
Step 1: Fill jug B     -> (0, 3)
Step 2: Pour B into A  -> (3, 0)
Step 3: Fill jug B     -> (3, 3)
Step 4: Pour B into A  -> (4, 2)

Target reached in 4 steps.
```

Result

The Water Jug problem was solved using BFS and 2 litres were measured in 4 steps.

Experiment 4: Missionaries and Cannibals Problem

Course Outcome: **CO1**

Aim

To write a Python program to carry 3 missionaries and 3 cannibals across a river using a boat that holds at most 2 people, without missionaries ever being outnumbered, using Breadth First Search.

Software / Requirements

Python 3 (any version 3.6 or later) with the standard library only (collections.deque). Any editor or IDE such as IDLE, VS Code or PyCharm.

Theory

Three missionaries and three cannibals are on the left bank of a river with one boat that carries one or two people. If cannibals outnumber missionaries on either bank (and at least one missionary is there), the missionaries are eaten, so such states are illegal. The boat needs at least one person to move.

A state is (M, C, B): missionaries on the left bank, cannibals on the left bank, and the boat side (1 = left, 0 = right). The start state is (3, 3, 1) and the goal state is (0, 0, 0). Each boat trip is an operator, and illegal states are rejected.

BFS searches the legal states level by level and so returns the solution with the fewest crossings.

Algorithm

1. Start with state (3, 3, 1) in a queue.
2. Remove a state. If it is (0, 0, 0), print the path and stop.
3. Try every boat load of 1 or 2 people (missionaries and/or cannibals) in the direction of the boat.
4. Compute the new state and keep it only if both banks are safe and it was not visited.
5. Add accepted states to the queue with the path so far. Repeat from step 2.

Procedure

1. Open a Python editor / IDE and create a new file named **exp4_missionaries.py**.
2. Type the program given below and save the file.
3. Open a terminal or command prompt in the folder containing the file.
4. Run the program with the command **python exp4_missionaries.py**.
5. Observe the output, verify that it satisfies the problem conditions and note the result.

Program - Experiment 4

```
# Experiment 4: Missionaries and Cannibals problem using BFS
# A state is (m, c, boat):
#   m = missionaries on the left bank, c = cannibals on the left bank
#   boat = 1 if the boat is on the left bank, 0 if it is on the right bank

start = (3, 3, 1)
goal = (0, 0, 0)


def valid(m, c):
    # numbers must be between 0 and 3 on both banks
    if m < 0 or m > 3 or c < 0 or c > 3:
        return False
    mr = 3 - m                      # missionaries on the right bank
    cr = 3 - c                      # cannibals on the right bank
    # missionaries must not be outnumbered on a bank that has missionaries
    if m > 0 and c > m:
        return False
    if mr > 0 and cr > mr:
        return False
    return True


def next_states(state):
    m = state[0]
    c = state[1]
    boat = state[2]
    result = []
    # the boat carries 1 or 2 people: (missionaries, cannibals)
    loads = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
    for dm, dc in loads:
        if boat == 1:               # boat goes from left to right
            new = (m - dm, c - dc, 0)
        else:                       # boat comes from right to left
            new = (m + dm, c + dc, 1)
        if valid(new[0], new[1]):
            result.append((dm, dc, new))
    return result


def solve():
    queue = [[start, []]]           # each item: [state, steps so far]
    visited = [start]

    while len(queue) > 0:
        item = queue.pop(0)
        state = item[0]
        steps = item[1]

        if state == goal:
            return steps

        for dm, dc, new in next_states(state):
            if new not in visited:
                visited.append(new)
                queue.append([new, steps + [(state, dm, dc, new)]])
    return None


# ---------------- main program ----------------
print("State = (Missionaries left, Cannibals left, Boat side)")
print("")
print("Start: " + str(start))

steps = solve()
n = 1
for old, dm, dc, new in steps:
    if old[2] == 1:
        direction = "left to right"
    else:
        direction = "right to left"
    print("Step " + str(n) + ": " + str(dm) + " missionary, " + str(dc) +
          " cannibal, " + direction + " -> " + str(new))
    n = n + 1

print("")
print("All people crossed safely in " + str(len(steps)) + " crossings.")
```

Output

```
State = (Missionaries left, Cannibals left, Boat side)

Start: (3, 3, 1)
Step 1: 0 missionary, 2 cannibal, left to right -> (3, 1, 0)
Step 2: 0 missionary, 1 cannibal, right to left -> (3, 2, 1)
Step 3: 0 missionary, 2 cannibal, left to right -> (3, 0, 0)
Step 4: 0 missionary, 1 cannibal, right to left -> (3, 1, 1)
Step 5: 2 missionary, 0 cannibal, left to right -> (1, 1, 0)
Step 6: 1 missionary, 1 cannibal, right to left -> (2, 2, 1)
Step 7: 2 missionary, 0 cannibal, left to right -> (0, 2, 0)
Step 8: 0 missionary, 1 cannibal, right to left -> (0, 3, 1)
Step 9: 0 missionary, 2 cannibal, left to right -> (0, 1, 0)
Step 10: 1 missionary, 0 cannibal, right to left -> (1, 1, 1)
Step 11: 1 missionary, 1 cannibal, left to right -> (0, 0, 0)

All people crossed safely in 11 crossings.
```

Result

The Missionaries and Cannibals problem was solved using BFS. All six people crossed safely in the minimum of 11 crossings.